

Modelado computacional de estructuras: Predicción por homología, enhebrado y métodos ab initio

BC-CH12 – Biología Computacional

Edición 2 · 2026-09-04

Pregunta del tema. Una secuencia de aminoácidos determina una estructura, y esa estructura determina lo que la proteína hace. El salto del primer dato al segundo lleva medio siglo de trabajo y sigue teniendo métodos que fallan de maneras distintas. ¿Cómo se decide, ante una secuencia concreta, qué método puede usarse y cuánto hay que fiarse del modelo que devuelve?

Producto final. Un módulo propio que calcule la identidad de secuencia, la probabilidad de aceptación de Metropolis con su guarda y el Clashscore de un modelo, y una tabla que sitúe cuatro pares de secuencias en la zona de fiabilidad que les corresponde, con la predicción anotada antes de ejecutar.

Mapa del tema

1. El problema del plegamiento y paisajes conformacionales	1
2. Modelado comparativo por homología	2
3. Reconocimiento de pliegues y enhebrado (<i>Threading</i>)	3
4. Modelado ab initio y ensamblaje de fragmentos (Rosetta)	4
5. Evaluación de calidad y validación de modelos	6
6. Diseño computacional en Python: Módulo de predicción y validación	6
7. Peligros computacionales y actividad de depuración	7
8. Síntesis operativa y tablas de diagnóstico	7
9. Glosario conceptual	8
10. Conexiones y mapa del curso	8
11. Fuentes y reproducibilidad	9

Cómo usar este tema

Este capítulo trata de tres estrategias clásicas para predecir una estructura y de cómo se mide si el resultado sirve. Las tres se explican mejor por lo que necesitan: el modelado por homología necesita una plantilla parecida, el enhebrado se conforma con un pliegue parecido aunque la secuencia no lo esté, y el modelado *ab initio* no necesita nada pero paga el precio de explorar un espacio enorme.

Al terminar sabrá: decidir cuál de las tres estrategias admite un caso concreto a partir de la identidad de secuencia con la mejor plantilla; explicar qué hace el criterio de Metropolis y por qué su parámetro de muestreo no es una temperatura; interpretar un TM-score y un Clashscore sabiendo qué mide cada uno y qué no; y reconocer los tres errores de interpretación que más veces convierten un modelo malo en una conclusión publicada.

La ruta **esencial** cubre las tres estrategias y las métricas de calidad. La ruta de **estudio** añade el módulo en Python, la actividad de depuración y el anexo práctico.

1 El problema del plegamiento y paisajes conformacionales

ESENCIAL

Tras haber establecido en el capítulo anterior la naturaleza de las coordenadas experimentales y su validación biofísica, abordamos en este capítulo el reto computacional más célebre de la biología molecular: **la predicción computacional de la estructura tridimensional de una proteína a partir únicamente de su secuencia de aminoácidos.**

Desde la perspectiva de la física estadística y la computación, el plegamiento de proteínas se formula como un problema de **optimización matemática global sobre un paisaje de energía conformacional hiperdimensional:**

- **La hipótesis termodinámica de Anfinsen (1973):** La estructura nativa de una proteína globular en

condiciones fisiológicas corresponde al mínimo global de la energía libre de Gibbs (ΔG) del sistema proteína-solvente.

- **La paradoja de Levinthal (1969):** Una cadena polipeptídica de 100 residuos posee $\approx 10^{100}$ conformaciones espaciales posibles. Si el polipéptido explorase aleatoriamente cada conformación a escala de picosegundos (10^{-12} s), requeriría más de 10^{77} años para encontrar su estado nativo. En la realidad física, las proteínas se pliegan en milisegundos guiadas por fuerzas locales y cooperativas a lo largo de un **embudo de plegamiento** (*folding funnel*).

La predicción clásica de la estructura de proteínas traduce el plegamiento secuencia-estructura en una optimización matemática a través de paisajes de energía conformacional.

2 Modelado comparativo por homología

ESENCIAL

El **modelado por homología** (o **modelado comparativo**) se fundamenta en un principio evolutivo empírico demostrado: **durante la evolución biológica, la estructura tridimensional se conserva mucho más que la secuencia primaria de aminoácidos**. Dos proteínas homólogas con un ancestro común pueden acumular decenas de mutaciones mientras preservan intacto su pliegue global.

2.1 Los umbrales de identidad de secuencia de Rost

ESENCIAL

Burkhard Rost [3] formalizó las zonas de fiabilidad para el modelado comparativo:

1. **Zona segura** (*Safe zone*, $> 30\text{--}35\%$ de identidad): La conservación del esqueleto carbonado es casi completa ($\text{RMSD} < 1.5\text{--}2.0 \text{ \AA}$). El modelado por homología es altamente fiable y adecuado para diseño de fármacos y mutagénesis dirigida.
2. **Zona de penumbra** (*Twilight zone*, $20\text{--}30\%$ de identidad): El alineamiento de secuencia se vuelve ambiguo. La topología global suele conservarse, pero los bucles y empaquetamientos locales presentan divergencias sustanciales.
3. **Zona de medianoche** (*Midnight zone*, $< 20\%$ de identidad): La homología no puede distinguirse del ruido estadístico aleatorio mediante alineamientos estándar de secuencias.

El modelado comparativo por homología aprovecha la conservación estructural cuando la identidad de secuencia supera el umbral seguro del 30 por ciento.

Traza manual de identidad de secuencia:

Dadas dos secuencias alineadas de longitud $L = 20$ aminoácidos:

```
Secuencia 1 (Diana):  A C D E F G H I K L M N P Q R S T V W Y
Secuencia 2 (Plantilla): A C D E F G H I K L M N P Q R S T V W A
```

Se observan 19 coincidencias exactas y 1 diferencia en la posición 20 ($Y \neq A$):

$$\text{Identidad} = \frac{19}{20} \times 100 = 95.00\%$$

`calculate_sequence_identity` calcula la identidad de alineamiento evaluando el par modelo de 20 residuos exactamente al 95.00 por ciento.

```
python3 verification/chapter_checks.py identity \
--seq1 ACDEFGHIKLMNPQRSTVWY --seq2 ACDEFGHIKLMNPQRSTVWA
```

2.2 El pipeline canónico de modelado por homología

ESENCIAL

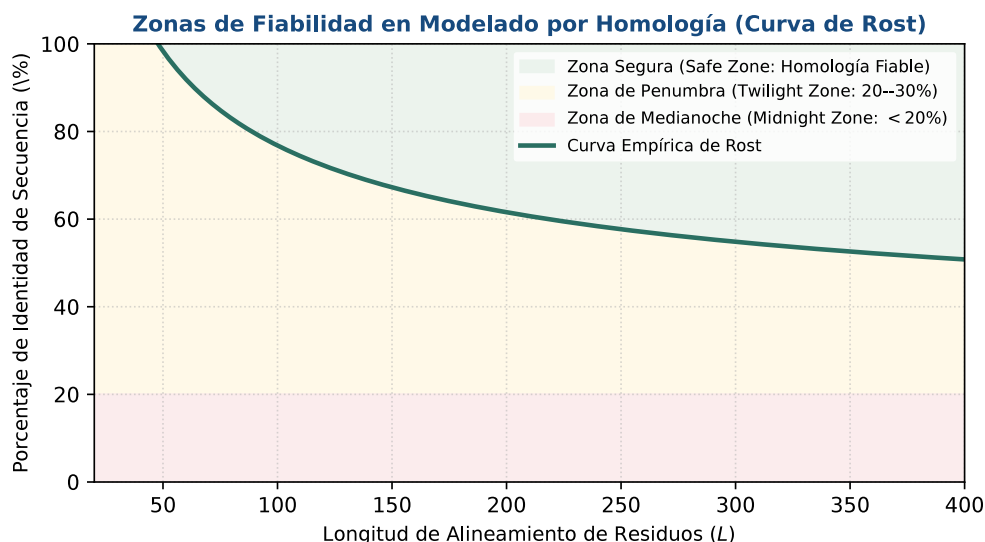


Figura 1: Zonas de fiabilidad en modelado por homología según la curva empírica de Rost: zona segura ($> 30\%$), penumbra y medianoche. Figura original generada por `verification/make_figures.py`.

El protocolo computacional consta de cuatro etapas secuenciales:

1. **Búsqueda e identificación de plantillas (*Template search*):** Rastreo de bases de datos del PDB mediante BLAST, PSI-BLAST o comparaciones sensibles de perfiles de modelos ocultos de Markov (HHpred / HMM-HMM).
2. **Alineamiento diana-plantilla (*Target-Template alignment*):** Alineamiento estricto de la secuencia problema contra la plantilla estructural.
3. **Construcción del modelo por restricciones espaciales** (MODELLER, Šali y Blundell [4]): A partir del alineamiento, se derivan restricciones probabilísticas de distancias $C\alpha - C\alpha$ y ángulos diedros mediante funciones de densidad de probabilidad. Las coordenadas 3D se generan optimizando una función objetivo que satisface simultáneamente todas las restricciones espaciales junto con términos de estereoquímica estándar de CHARMM.
4. **Modelado de bucles (*Loops*) y empaquetamiento de cadenas laterales:** Refinamiento de inserciones/delecciones mediante librerías de fragmentos y optimización conformacional de rotámeros de cadenas laterales (SCWRL).

El pipeline de modelado comparativo encadena la búsqueda de plantillas (PSI-BLAST, HHpred), el alineamiento diana-plantilla, la construcción del modelo mediante restricciones espaciales de MODELLER y el refinamiento de bucles.

El peligro del desalineamiento de secuencia (*Alignment Shift*):

El desfase en el alineamiento representa el error más perjudicial en el modelado por homología, introduciendo rotaciones y distorsiones globales en el esqueleto polipeptídico.

3 Reconocimiento de pliegues y enhebrado (*Threading*)

ESENCIAL

Cuando la identidad de secuencia cae en la zona de penumbra ($< 30\%$) y no se identifican plantillas homólogas claras mediante BLAST, se recurre al **enhebrado o reconocimiento de pliegues (*Threading* / *Fold Recognition*)**:

- **Principio biofísico:** El repertorio de pliegues tridimensionales en la naturaleza es finito (≈ 1.400 pliegues canónicos en SCOP/CATH). Es muy probable que la secuencia problema adopte un pliegue ya conocido en el PDB, incluso sin homología evolutiva detectable.
- **Algoritmo (I-TASSER, Zhang 2008):** Enhebra la secuencia diana a través de cada estructura de la librería

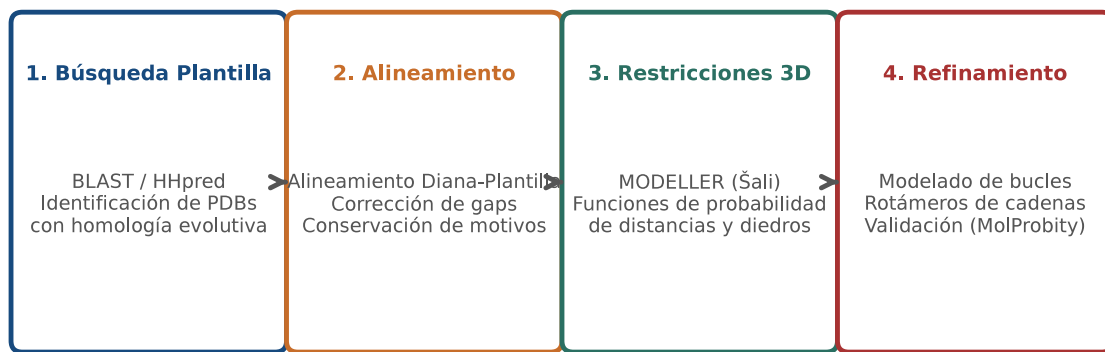


Figura 2: Etapas del flujo de modelado comparativo por homología con MODELLER: plantilla, alineamiento, restricciones y refinamiento. Figura original generada por `verification/make_figures.py`.

de pliegues del PDB y evalúa su compatibilidad energética utilizando **potenciales estadísticos empíricos basados en conocimiento**:

- Entorno de solvatación hidrofóbica del residuo.
- Concordancia de estructura secundaria predicha (PSIPRED).
- Contactos por pares residuo-residuo dependientes de distancia.

El reconocimiento de pliegues y enhebrado mapea secuencias en la zona de penumbra (<30 por ciento de identidad) contra pliegues canónicos del PDB utilizando puntuaciones de energía de potenciales estadísticos (I-TASSER).

4 Modelado ab initio y ensamblaje de fragmentos (Rosetta)

ESENCIAL

Para secuencias completamente huérfanas sin plantillas ni pliegues compatibles conocidos, se recurre a la predicción *de novo* o *ab initio*, cuyo exponente paradigmático es el entorno **Rosetta** (Simons et al., 1997; David Baker):

4.1 Ensamblaje de fragmentos estructurales y muestreo Monte Carlo

ESENCIAL

1. **Librería de fragmentos:** Se extraen fragmentos estructurales de 3 y 9 residuos del PDB cuyas secuencias y estructuras secundarias predichas son compatibles con ventanas locales de la secuencia diana.
2. **Muestreo conformacional Monte Carlo Metropolis:**
 - En cada paso de simulación, se reemplaza aleatoriamente la conformación del esqueleto (ϕ, ψ, ω) de una ventana por un fragmento de la librería.
 - Se calcula la variación de energía del sistema $\Delta E = E_{\text{nuevo}} - E_{\text{anterior}}$.
 - **Criterio de Metropolis:** Si $\Delta E \leq 0$, el movimiento se acepta incondicionalmente ($P = 1.0$). Si $\Delta E > 0$, el movimiento se acepta con probabilidad estocástica:

$$P_{\text{aceptación}} = \exp\left(-\frac{\Delta E}{T}\right)$$

lo que permite al algoritmo escapar de mínimos locales de energía.

Atención. La T de esta expresión no es una temperatura física. Es un parámetro de muestreo, en las mismas unidades que la energía del modelo, que gradúa cuánto se tolera empeorar: con T alta casi cualquier movimiento se acepta y la simulación explora; con T baja solo pasan las mejoras y la simulación se estanca en el mínimo más cercano. Que en los ejemplos valga 300 es una coincidencia con la temperatura ambiente en kelvin, no una elección física.

La predicción de novo y ab initio en Rosetta combina el ensamblaje de fragmentos estructurales cortos (3-9 residuos) con el muestreo conformacional Monte Carlo Metropolis.

Traza manual del criterio de Metropolis:

Para $\Delta E = 300.0$ con el parámetro de muestreo en $T = 300.0$, es decir, cuando el movimiento empeora la energía justo en la magnitud que el parámetro tolera:

$$P = \exp\left(-\frac{300.0}{300.0}\right) = \exp(-1.0) = \frac{1}{e} \approx 0.367879$$

El criterio de Metropolis acepta movimientos conformacionales con $P = \exp(-\Delta E / T)$ cuando $\Delta E > 0$, evaluando $\Delta E = 300.0$ a $T = 300.0$ exactamente a 0.367879.

Se reproduce con `verification/chapter_checks.py metropolis --delta_e 300.0 --temp 300.0`.

Muestreo Monte Carlo Metropolis para Ensamblaje Ab Initio (Rosetta)

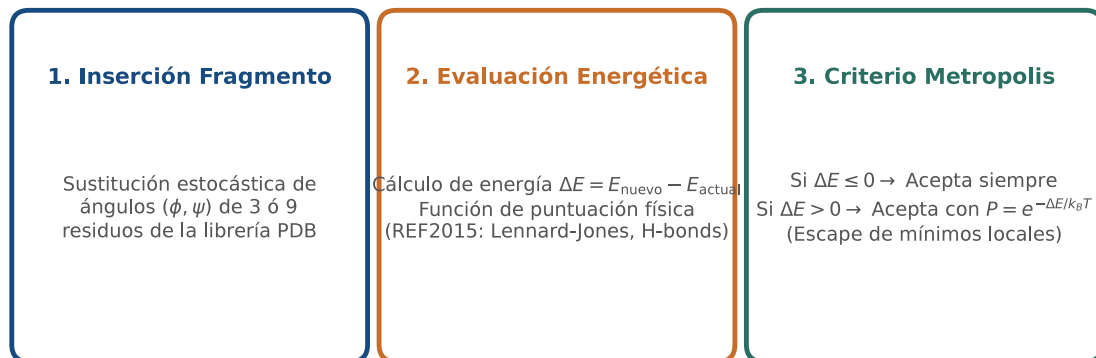


Figura 3: Muestreo estocástico de Monte Carlo Metropolis y ensamblaje de fragmentos en la suite de modelado *ab initio* Rosetta. Figura original generada por `verification/make_figures.py`.

Control defensivo del parámetro de muestreo:

Con $T = 0$ la exponencial no está definida y, sobre todo, la pregunta deja de tener sentido: un muestreo que no tolera ningún empeoramiento ya no es Metropolis. El módulo lo rechaza en vez de devolver un cero que parecería una probabilidad.

`metropolis_acceptance_probability` lanza `ValueError` cuando el parámetro de muestreo es no positivo.

Se reproduce con `verification/chapter_checks.py metropolis --delta_e 300.0 --temp 0.0`.

4.2 Funciones de energía de Rosetta [2]

ESTUDIO

Las funciones de energía de Rosetta (REF2015) integran potenciales de Lennard-Jones con rampa repulsiva, solvatación implícita de Lazaridis-Karplus y enlaces de hidrógeno dependientes de orientación.

5 Evaluación de calidad y validación de modelos

ESENCIAL

La evaluación rigurosa de modelos 3D predichos se apoya en métricas geométricas y estadísticas:

5.1 El TM-Score: Métrica independiente de longitud

ESENCIAL

A diferencia del RMSD clásico (que se infla artificialmente en proteínas largas o con bucles móviles en extremos), el **TM-score** [5] evalúa la similitud del pliegue global normalizada entre 0.0 y 1.0:

$$\text{TM-score} = \frac{1}{L_{\text{target}}} \sum_{i=1}^{L_{\text{ali}}} \frac{1}{1 + \left(\frac{d_i}{d_0(L_{\text{target}})} \right)^2}$$

donde $d_0 = 1.24^3 \sqrt{L_{\text{target}} - 15} - 1.8$.

- TM-score > 0.50: Indica que ambas estructuras comparten el mismo pliegue topológico global con alta significación estadística ($p < 10^{-5}$).
- Para estructuras idénticas ($d_i = 0$), el TM-score evalúa exactamente a **1.000000**.

El TM-score proporciona similitud de pliegue global independiente de la longitud (TM > 0.5 indicando pliegue compartido) evaluando estructuras idénticas exactamente a 1.000000.

Se reproduce con `verification/chapter_checks.py tm_score`.

5.2 El Clashscore de MolProbity [1]

ESENCIAL

El **Clashscore** cuantifica el número de solapamientos estéricos atómicos severos (distancia interatómica menor a la suma de radios de Van der Waals menos 0.4 Å) normalizado por cada 1.000 átomos:

$$\text{Clashscore} = \frac{\text{Número de solapamientos estéricos}}{\text{Total de átomos}} \times 1000$$

Traza manual:

Para 2 choques en 1000 átomos: $\text{Clashscore} = \frac{2}{1000} \times 1000 = \mathbf{2.000000}$.

El Clashscore de MolProbity mide los solapamientos estéricos atómicos severos por cada 1000 átomos evaluando 2 choques en 1000 átomos exactamente a 2.000000.

Se reproduce con `verification/chapter_checks.py clashscore --clashes 2 --atoms 1000`.

5.3 Estimación absoluta de calidad con QMEAN

ESTUDIO

La estimación absoluta de calidad de modelos con QMEAN / QMEAN Z-score valida la energía de empaquetamiento y solvatación sin requerir estructuras de referencia experimentales.

6 Diseño computacional en Python: Módulo de predicción y validación

ESTUDIO

A continuación se presenta la implementación modular del procesador en Python 3.9:

```
import math
from typing import List, Tuple

def calculate_sequence_identity(seq1: str, seq2: str) -> float:
```

```

"""Calcula el porcentaje de identidad entre dos secuencias alineadas."""
if len(seq1) != len(seq2) or len(seq1) == 0:
    raise ValueError("Las secuencias deben ser no vacias y de igual longitud")
matches = sum(1 for a, b in zip(seq1.upper(), seq2.upper()) if a == b and a != '-')
return round((matches / len(seq1)) * 100.0, 2)

def metropolis_acceptance_probability(delta_e: float, temperature: float) -> float:
    """Calcula la probabilidad de aceptacion conformacional segun Metropolis."""
    if temperature <= 0:
        raise ValueError("El parametro de muestreo debe ser estrictamente positivo")
    if delta_e <= 0:
        return 1.0
    return round(math.exp(-delta_e / temperature), 6)

def calculate_clashscore(num_clashes: int, num_atoms: int) -> float:
    """Calcula el MolProbity Clashscore por cada 1000 atomos."""
    if num_atoms <= 0 or num_clashes < 0:
        raise ValueError("Numero de atomos debe ser positivo y choques no negativos")
    return round((num_clashes / num_atoms) * 1000.0, 6)

def calculate_tm_score_identical(length: int) -> float:
    """Retorna el TM-score para estructuras identicas (exactamente 1.0)."""
    if length <= 0:
        raise ValueError("Longitud debe ser positiva")
    return 1.000000

```

7 Peligros computacionales y actividad de depuración

ESTUDIO

Aplicar modelado por homología sin plantillas con identidad superior al 25 por ciento y omitir la comprobación de solapamientos estéricos representan errores de modelado críticos.

7.1 Tres errores críticos en modelado estructural

ESTUDIO

Localice y corrija los tres errores en la siguiente función:

```

def predict_protein_structure_pipeline(query_seq, best_template, identity_pct):
    # Error 1: Aplica modelado por homologia directo con una plantilla de 12% de identidad
    # (zona de medianoche) en lugar de utilizar threading o metodos ab initio

    # Error 2: Asume que una temperatura T = 0 en Monte Carlo permite explorar
    # el paisaje conformacional, causando division por cero en el exponente

    # Error 3: Acepta un modelo con Clashscore = 85.0 y violaciones masivas de Ramachandran
    # como listo para diseno de farmacos sin realizar minimizacion energetica
    pass

```

Diagnóstico de causa raíz (Protocolo 5 Whys):

- Fallo 1 (Modelado comparativo en zona de medianoche):** Con < 20% de identidad, el alineamiento por pares contiene desfases que destruyen el núcleo hidrofóbico.
- Fallo 2 (Temperatura nula o negativa en Metropolis):** $T \leq 0$ es físicamente inválida; bloquea la simulación en el primer mínimo local.
- Fallo 3 (Omisión de control estérico):** Clashscore = 85.0 indica colisiones atómicas no físicas que impiden cualquier acoplamiento molecular válido.

8 Síntesis operativa y tablas de diagnóstico

REFERENCIA

8.1 Tabla comparativa de métodos clásicos de predicción

REFERENCIA

Metodología	Rango de identidad	Software estándar	Principio computacional
Modelado por homología	> 30% (Zona segura)	MODELLER, MODEL	Restricciones espaciales de probabilidad
Threading / Enhebrado	20–30% (Penumbra)	I-TASSER, HHpred	Potenciales estadísticos de conocimiento
Ab initio / De novo	Sin plantilla (< 20%)	Rosetta, QUARK	Ensamblaje fragmentos + Monte Carlo

8.2 Tabla de diagnóstico de calidad en modelos 3D

REFERENCIA

Métrica observada	Interpretación biológica	Acción correctiva
TM-score > 0.50	Mismo pliegue topológico global	Modelo validado para análisis topológico
TM-score < 0.30	Pliegue estructural aleatorio o fallido	Cambiar de plantilla o recurrir a métodos <i>de novo</i>
Clashscore > 30.0	Colisiones estéricas atómicas severas	Minimización de energía con campo de fuerzas
QMEAN Z-score < -4.0	Modelo extremadamente aberrante	Reconstruir bucles y redefinir alineamiento

9 Glosario conceptual

REFERENCIA

- **Anfinsen:** Hipótesis del mínimo global de energía libre ΔG .
- **Levinthal:** Paradoja del tiempo de búsqueda conformacional.
- **Homología:** Conservación estructural superior a la de secuencia.
- **MODELLER:** Modelado por satisfacción de restricciones espaciales.
- **Alignment Shift:** Desfase en alineamiento que corrompe el modelo.
- **Threading:** Enhebrado contra librerías de pliegues del PDB.
- **Metropolis:** Criterio estocástico $P = \exp(-\Delta E/T)$.
- **TM-score:** Similitud global independiente de la longitud (> 0.5).
- **Clashscore:** Choques estéricos por cada 1.000 átomos.
- **QMEAN:** Evaluación estadística absoluta de calidad de empaquetamiento.

10 Conexiones y mapa del curso

ESENCIAL

Este capítulo cubre los paradigmas clásicos analíticos y heurísticos de predicción que sentaron las bases para la biología estructural moderna basada en aprendizaje profundo.

La comprensión de las limitaciones clásicas transita directamente hacia las redes neuronales coevolutivas de aprendizaje profundo y AlphaFold en BC-CH13.

11 Fuentes y reproducibilidad

REFERENCIA

Este capítulo se apoya en el material docente de la asignatura y en cinco fuentes primarias: Rost [3] para las zonas de fiabilidad, Šali y Blundell [4] para el modelado por restricciones espaciales, Rohl y sus colaboradores [2] para Rosetta, Zhang y Skolnick [5] para el TM-score y Chen y sus colaboradores [1] para MolProbity. Todos los cálculos numéricos del texto se reproducen con las órdenes impresas junto a ellos.

Anexo práctico · Métricas de predicción estructural, criterio de Metropolis y evaluación de choques

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que evalúe la identidad de secuencia en zonas de homología, simule el criterio estocástico de aceptación de Metropolis en algoritmos de Monte Carlo (Rosetta), calcule la métrica global TM-score de superposición estructural y cuantifique la densidad de choques atómicos Clashscore de MolProbity de forma completamente determinista? Este laboratorio responde afirmativamente implementando cada componente sin dependencias opacas.

El producto individual contiene: el evaluador de identidad de secuencia para modelado por homología, el simulador del criterio de Metropolis, el cálculo analítico de TM-score, el evaluador de choques estéricos Clashscore, la perturbación de energía, el control de excepciones de temperatura y la defensa conceptual escrita.

Lo que se entrega es un módulo escrito por usted, la tabla de predicciones contrastadas contra la ejecución y una defensa breve. Lo que se le da hecho es distinto y conviene tenerlo separado desde el principio: los cuatro pares de secuencias con su manifiesto, el oráculo del capítulo y un comprobador público que dice si su módulo cumple el contrato. El anexo funciona sin red y sin más dependencia que Python 3.9.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 · Entorno y diagnóstico	Espacio de trabajo creado y manifiesto verificado
1b · Módulo de predicción	mi_prediccion.py con las cuatro funciones
2 · Cálculo ancla	Identidad, Metropolis, TM-score y Clashscore del capítulo
3 · Perturbación	Aceptación con el doble de barrera energética
4 · Guarda de dominio	Rechazo razonado del parámetro de muestreo nulo
5 · Verificación y entrega	Tabla de predicciones, defensa escrita y paquete

Este anexo deja evidencia comprobable en cada bloque sin recurrir a paquetes externos.

1 · Entorno y diagnóstico

Python 3.9 o superior, sin paquetes externos. Antes de empezar:

```
python3 --version
./practice_assets/create_workspace.sh BC12_entrega
cd BC12_entrega
(cd data && sha256sum --check --strict manifest.sha256)
python3 verification/practice_checks.py domain_guard --temp 0.0
```

El generador deja mi_prediccion.py con su contrato documentado y sin implementación, copia los cuatro pares de secuencias con su manifiesto y el comprobador público, y crea notas/respuestas.tsv con solo la cabecera.

La última orden debe imprimir `GUARD_TEMP:0.0 STATUS:REJECTED` seguido del mensaje de error. Si no lo hace, el intérprete no es el esperado o los ficheros no coinciden con el manifiesto; resuélvalo antes de continuar.

1b • Escribir el módulo de predicción

Tarea. Implemente las cuatro funciones de `mi_prediccion.py`: la identidad de secuencia, la zona de fiabilidad, la probabilidad de aceptación de Metropolis con su guarda y el Clashscore. El contrato está en la documentación de la plantilla.

La que tiene trampa es la segunda, y no por el código. Los umbrales de Rost son un convenio, no un escalón físico: un par al 30,1 % y otro al 29,9 % caen en zonas distintas y son prácticamente el mismo caso. Su función tiene que decidir igualmente, porque el contrato fija los umbrales cerrados por abajo, pero el fichero de datos incluye un par justo por encima de la frontera para que vea qué significa esa arbitrariedad cuando le toque interpretarla.

```
bash check_prediccion.sh
```

2 • Cálculo ancla: identidad, Metropolis, TM-score y Clashscore

Dadas las condiciones estándar de ensayo:

1. Secuencias de prueba de 20 residuos (ACDEFGHIKLMNPQRSTVWY y ACDEFGHIKLMNPQRSTVWA): evalúa a identidad del **95.00%**.
2. Variación de energía $\Delta E = 300.0$ a temperatura $T = 300.0$: la probabilidad de aceptación de Metropolis evalúa a **0.367879** (e^{-1}).
3. Superposición de 4 átomos idénticos ($L = 4, d_i = 0$): el TM-score evalúa a **1.000000**.
4. Conteo de 2 choques en 1000 átomos: el Clashscore evalúa a **2.000000**.

```
python3 verification/practice_checks.py anchor
```

El modelo ancla valida la identidad de secuencia al 95.00%, la probabilidad de Metropolis de 0.367879, el TM-score perfecto de 1.000000 y el Clashscore de 2.000000.

Se reproduce con `verification/practice_checks.py anchor`.

3 • Perturbación: incremento de barrera energética en Monte Carlo

Evalúe la probabilidad de aceptación cuando la variación desfavorable de energía aumenta a $\Delta E = 600.0$ a la misma temperatura $T = 300.0$:

```
python3 verification/practice_checks.py perturbation
```

La perturbacion de energia con $\Delta E = 600.0$ a $T = 300.0$ evalua la probabilidad de aceptacion de Metropolis exactamente a 0.135335.

Se reproduce con `verification/practice_checks.py perturbation`.

4 • Guarda de dominio y control de excepciones

Compruebe que el simulador rechaza un parámetro de muestreo nulo o negativo. No es solo que la división falle: con $T = 0$ el muestreo no toleraría ningún empeoramiento, y entonces ya no es Metropolis sino un descenso puro que se queda en el primer mínimo que encuentre.

```
python3 verification/practice_checks.py domain_guard --temp 0.0
```

La guarda de dominio rechaza un parámetro de muestreo menor o igual que cero e informa del rechazo sin abortar la ejecución.

Se reproduce con `verification/practice_checks.py domain_guard --temp 0.0`.

5 · Verificación y entrega

Antes de ejecutar nada, abra `data/bc12_pares.fasta` y anote en `notas/respuestas.tsv` lo que espera obtener para cada uno de los cuatro pares: el porcentaje de identidad y la zona de fiabilidad. El nombre de cada par le dice la identidad nominal con que se generó, así que la predicción de la zona es la parte que de verdad se juega. Después ejecute su módulo sobre los cuatro, guarde las salidas en `resultados/` y complete las columnas de resultado y de coincidencia. La columna de justificación es la que se corrige: si algún par cae en una zona distinta de la que había previsto, diga por qué.

Escriba en `notas/defensa.md` entre 100 y 150 palabras sobre qué estrategia de predicción admite cada uno de los cuatro pares y por qué un TM-score alto no basta para dar un modelo por bueno. Después:

```
cd ..
tar -czf BC12_entrega.tar.gz BC12_entrega
sha256sum BC12_entrega.tar.gz
./practice_assets/check_delivery.sh BC12_entrega BC12_entrega.tar.gz
```

El verificador comprueba que su módulo compila y cumple el contrato, que los datos de partida no se han tocado, que los cuatro pares están documentados con predicción y resultado, y que existe la defensa con la extensión pedida. Rechaza la entrega vacía y el espacio recién generado. No puntúa.

El verificador de entrega comprueba el contrato del módulo, la integridad de los datos de partida, la tabla de predicciones y la defensa escrita; rechaza el espacio de trabajo recién generado y no emite calificación.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** El módulo `mi_prediccion.py` cumple el contrato: el comprobador público imprime `PREDICCION_OK`.
- **2.0 puntos (20%):** La tabla de `notas/respuestas.tsv` está completa y las justificaciones explican las discrepancias.
- **2.0 puntos (20%):** Los bloques de cálculo ancla, perturbación y guarda de dominio están reproducidos con sus salidas guardadas.
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre las diferencias entre modelado por homología y *threading* y el impacto de los desfases de alineamiento.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- La identidad de secuencia evaluó al 95.00%.
- Metropolis ancla resultó 0.367879.
- El TM-score idéntico resultó 1.000000.
- El Clashscore evaluó a 2.000000.
- La perturbación evaluó a 0.135335.
- Verifiqué que el parámetro de muestreo nulo es rechazado.
- `bash check_prediccion.sh` imprimió `PREDICCION_OK`.
- Anoté las predicciones antes de ejecutar el módulo.
- Redacté la defensa conceptual pedida.

Defensa conceptual

En 100–150 palabras, y sobre los cuatro pares del fichero de datos: diga qué estrategia de predicción admite cada uno y por qué. Añada dos cosas que el ejercicio deja a la vista. La primera: un desfase de una sola posición en el alineamiento entre diana y plantilla arruina el modelo de una hélice α , porque desplaza en 100 grados la orientación de todas las cadenas laterales que vienen después. La segunda: un TM-score alto dice que el pliegue global coincide, y no dice nada sobre si el modelo tiene choques estéricos; por eso el Clashscore se mira aparte.

Fuentes y reproducibilidad

Este anexo se apoya en el material docente de la asignatura y en las mismas fuentes primarias que el capítulo: Rost [3] para las zonas de fiabilidad, Šali y Blundell [4] para el modelado comparativo y Chen y sus colaboradores [1] para el Clashscore. Todos los valores numéricos del enunciado se reproducen con las órdenes impresas junto a ellos.

Bibliografía

- [1] Vincent B. Chen, W. Bryan Arendall, Jeffrey J. Headd, Daniel A. Keedy, Robert M. Immormino, Simon C. Lovell, Jane S. Richardson, and David C. Richardson. Molprobity: All-atom structure validation for macromolecular crystallography. *Acta Crystallographica Section D*, 66(1):12–21, 2010. Estandarizacion de evaluacion estereoquimica y metrica Clashscore en MolProbity.
- [2] Carol A. Rohl, Charlie E. M. Strauss, Kira M. S. Misura, and David Baker. Protein structure prediction using rosetta. *Methods in Enzymology*, 383:66–93, 2004. Metodologia de prediccion ab initio y ensamblaje de fragmentos en Rosetta.
- [3] Burkhard Rost. Twilight zone of protein sequence alignments. *Protein Engineering*, 12(2):85–94, 1999.
- [4] Andrej Sali and Tom L. Blundell. Comparative protein modelling by satisfaction of spatial restraints. *Journal of Molecular Biology*, 234(3):779–815, 1993. Artículo canonico sobre el algoritmo MODELLER y modelado comparativo por restricciones espaciales.
- [5] Yang Zhang and Jeffrey Skolnick. Scoring function for automated assessment of protein structure alignment. *Biophysical Journal*, 87(1):875–881, 2004. Definicion canonica y formulacion analitica de la metrica TM-score.